

SOFTWARE TESTING ASSIGNMENT - PART 4

SRS Peer Review, Informal Reviews, Walkthroughs, and Static Testing

1. Download a sample SRS (Software Requirements Specification) document for a food delivery app (like Zomato) and perform a peer review by highlighting at least 5 unclear or ambiguous requirements.

A comprehensive peer review was performed on a sample Food Delivery/Zomato-style SRS. The following specific, measurable, and testable improvements have been identified to address ambiguous requirements:

Peer Review Findings

Requirement (Ambiguous)	Issue Found	Suggested Improvement
"The system should provide fast search results."	"Fast" is subjective and not measurable.	"Search results should be displayed within 2 seconds for 95% of searches."
"The application should be user-friendly."	"User-friendly" means different things to different people.	"A new user should be able to place an order in 5 steps or fewer."
"The system should send notifications instantly."	"Instantly" is unclear.	"Order notifications should be delivered within 10 seconds of the event."
"The app should support secure payments."	"Secure" is vague and not testable.	"All payment transactions must use HTTPS and comply with PCI-DSS standards."
"The restaurant list should load quickly."	"Quickly" is not measurable.	"Restaurant listings should load within 3 seconds on a 4G connection."

Additional Review Comments

1. Some requirements do not define performance limits.
2. Terms such as **fast**, **quickly**, **secure**, and **user-friendly** are subjective.
3. Non-functional requirements should include measurable values (response time, availability, security standards).
4. Acceptance criteria are missing for several features.
5. Error-handling requirements (e.g., payment failure, network failure) are not clearly specified.

2. Conduct an informal review of a friend's written test case set for a simple login feature (username and password), and write down 3 suggestions or improvements you would make.

Informal Review of Login Feature Test Cases

After reviewing the test cases for the Username and Password Login feature, the following key suggestions are made to increase test coverage and completeness:

Suggestion No.	Improvement Description
1	Add Negative Test Cases: Include scenarios such as invalid username, invalid password, blank username, and blank password to ensure the system handles incorrect inputs properly.
2	Define Clear Expected Results: Each test case should have a specific expected outcome. Example: "Error message 'Invalid Username or Password' should be displayed."
3	Include Validation and Boundary Value Conditions: Add test cases for minimum/maximum username and password lengths to verify restricted bounds and characters.

3. Simulate a walkthrough session by explaining the steps of a test case for 'Add to Cart' functionality (like Flipkart) to a classmate and note any questions or feedback they give you.

Walkthrough Session: "Add to Cart" Functionality (Flipkart-like App)

Test Case ID: TC_01 | **Test Scenario:** Add a Product to Cart

Preconditions:

- User is logged into the application.
- Product is available in stock.

Detailed Test Steps:

1. Open the shopping app.
2. Search for a product (e.g., "Wireless Mouse").
3. Open the product details page.
4. Click the **Add to Cart** button.
5. Navigate to the Cart page.
6. Verify that the selected product appears in the cart with the correct quantity and price.

Expected Result:

- The product is successfully added to the cart.
- The cart count icon increases by 1.
- The correct product details, quantity, and price are displayed in the cart.

Classmate's Questions and Feedback Log

Question / Feedback Received	Action Taken & Resolution
Is the user required to be logged in before adding a product to the cart?	Added an explicit precondition stating that the user should be logged in.
What happens if the product is out of stock?	Suggested isolating this behavior and creating a separate test case for out-of-stock validation.
Should we verify the cart count icon after adding the product?	Updated the expected results to include the direct validation of the cart count icon.
What quantity should be checked in the cart?	Clarified within the documentation that the default verification quantity should be 1.
Should the price in the cart match the product page price?	Added direct price verification benchmarks to the expected results template.

4. Identify 4 types of errors or issues that static testing (reviewing documents or code without execution) can catch in a test case document for an online movie booking system (like BookMyShow).

Static Testing Issues Found (Online Movie Booking System)

Error Type	Description	Example Context (BookMyShow-style)
1. Missing Requirements Coverage	Some requirements are completely unmapped or not covered by any test case in the document.	The core requirement " <i>User can cancel a movie ticket</i> " exists in the SRS, but no test case is written for it.
2. Ambiguous or Unclear Test Steps	Test steps are written vaguely and can be interpreted differently by different execution testers.	A step states " <i>Select a movie and proceed</i> " without specifying which interface, button, or layout screen to interact with.
3. Incorrect or Missing Expected Results	Expected results are absent, incomplete, or not clearly bounded for evaluation.	The expected result states " <i>Booking should work properly</i> " instead of defining specific confirmation screen alerts and generated tickets.
4. Inconsistent or Incorrect Test Data	The associated test data does not logically match the scenario workflow or contains data formatting errors.	A designated test case designed for invalid payment processing incorrectly uses a completely valid credit card profile as input test data.